

Лабораторная работа № 2 по курсу дискретного анализа: сбалансированные деревья

Выполнил студент группы М8О-208Б-22 МАИ *Немкова Анастасия*.

Условие

Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64} - 1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

+ **word 34** — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.

- **word** — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.

word — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».

! **Save /path/to/file** — сохранить словарь в бинарном компактном представлении на диск в файл, указанный параметром команды. В случае успеха, программа должна вывести «OK», в случае неудачи выполнения операции, программа должна вывести описание ошибки (см. ниже).

! **Load /path/to/file** — загрузить словарь из файла. Предполагается, что файл был ранее подготовлен при помощи команды Save. В случае успеха, программа должна вывести строку «OK», а загруженный словарь должен заменить текущий (с которым происходит работа); в случае неуспеха, должна быть выведена диагностика, а рабочий словарь должен остаться без изменений. Кроме системных ошибок, программа должна корректно обрабатывать случаи несовпадения формата указанного файла и представления данных словаря во внешнем файле. **Задача:** Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант:

1. AVL-дерево

Метод решения

Для реализации словаря используется AVL-дерево.

AVL-дерево - это бинарное дерево поиска, являющееся сбалансированным в следующем смысле: высота правого поддерева для любого узла дерева отличается от высоты его левого поддерева не более чем на единицу.

При разнице высот поддеревьев равной 2 или -2 используется балансировка дерева, для этого применяются две вспомогательные функции: правый и левый повороты дерева.

Вставка и удаление узлов происходит как и в обычном бинарном дереве, но в конце процедур выполняется балансировка.

Поиск в AVL-дерево полностью аналогичен поиску в бинарном дереве.

Для записи дерева в файл последовательно записывается размер ключа узла, сам ключ и значение узла, а также информацию о наличии левого и правого потомков.

При загрузке из файла происходит обратный процесс: считывается размер ключа, выделяется память для ключа, считывается сам ключ, затем значение узла, флаги наличия потомков, и, если потомок присутствует, рекурсивно загружается поддерево.

Словарь хранит пары ключ-значение. Ключом выступает строка до 256 символов, значение - беззнаковое число от 0 до $2^{64} - 1$

Описание программы

Для реализации словаря был создан класс AVL-дерева, каждый узел которого содержит информацию о ключе, значении и высоте узла, а также указатель на правого и левого потомка. Тип данных для значения ключа `unsigned long long`, позволяющий хранить любое значение до $2^{64} - 1$ символов. Ключ является строкой переменной длины до 256 символов, для её хранения был создан класс `TString`, содержащий информацию о размере и вместимости строки и массив элементов типа `char`. В классе AVL-дерева реализованы следующие методы:

- `unsigned char GetHeight(TNode* n)` - возвращает высоту дерева.
- `int BFactor(TNode* n)` - разница между высотами правого и левого поддерева данного узла.
- `void FixHeight(TNode* n)` - обновление высоты данного узла на основе высот его дочерних узлов.
- `TNode* RotateRight(TNode* n)` - правый поворот вокруг данного узла.
- `TNode* RotateLeft(TNode* n)` - левый поворот вокруг данного узла.
- `TNode* Balance(TNode* n)` - балансировка данного узла в соответствии с его высотой и фактором сбалансированности.
- `TNode* InsertNode(TNode* n, TNode* k)` - вставка узла.

- `TNode* FindMin(TNode* n)` - находит узел с минимальным ключом в дереве с корнем в данном узле.
- `TNode* RemoveMin(TNode* n)` - удаление узла с минимальным ключом в дереве с корнем в данном узле.
- `TNode* RemoveNode(TNode* n, const T k)` - удаление узла.
- `TNode* FindNode(TNode* n, const T k)` - нахождение узла с заданным ключом.
- `void Delete(TNode* n)` - удаление дерева.
- `void Save(TNode* n, std::ofstream of)` - сохранение дерева в файл.
- `TNode* Load(std::ifstream is)` - загрузка дерева из файла.

Дневник отладки

1. 19 апр 2024, 16:07:36 OL на 1 тесте

Решение: добавлена проверка для выхода из цикла, считающего данные из стандартного потока ввода, при достижении конца файла

2. 19 апр 2024, 16:11:42 WA на 15 тесте

Решение: изменено удаление узла дерева: вместо перемещения минимального узла дерева на место удаляемого, создаем новый узел на основе минимального и присоединяем к нему левое поддерево

3. 24 апр 2024, 00:47:55 TL на тесте 15

Решение: вместо приведения к нижнему регистру ключа при каждой операции сравнения, используем эту операцию сразу же при считывании значения ключа из стандартного потока ввода

Тест производительности

Для измерения производительности сравнивается время операций вставки, удаления и поиска реализованного класса AVL-дерева и стандартного контейнера `std::map`, в основе которого лежит красно-черное дерево, на одинаковых входных данных. Подсчет времени производился с помощью библиотеки `chrono`, которая позволяет фиксировать время в начале и конце выполнения сортировки. Первый тест состоит из 10^3 строк, второй из $5 * 10^3$ строк, третий из 10^4 , четвертый из $5 * 10^4$ и пятый из 10^5 строк.

На графиках представлены зависимости времени выполнения вставки, удаления и поиска узла от объёма входных данных. Сложность данных операций в AVL-дереве $O(\log(n))$, где n - количество узлов дерева. Как видно из графика, вставка и удаление элемента работают быстрее в `std::map`, что связано с необходимостью балансировки AVL-дерева каждый раз после совершения данных операций.

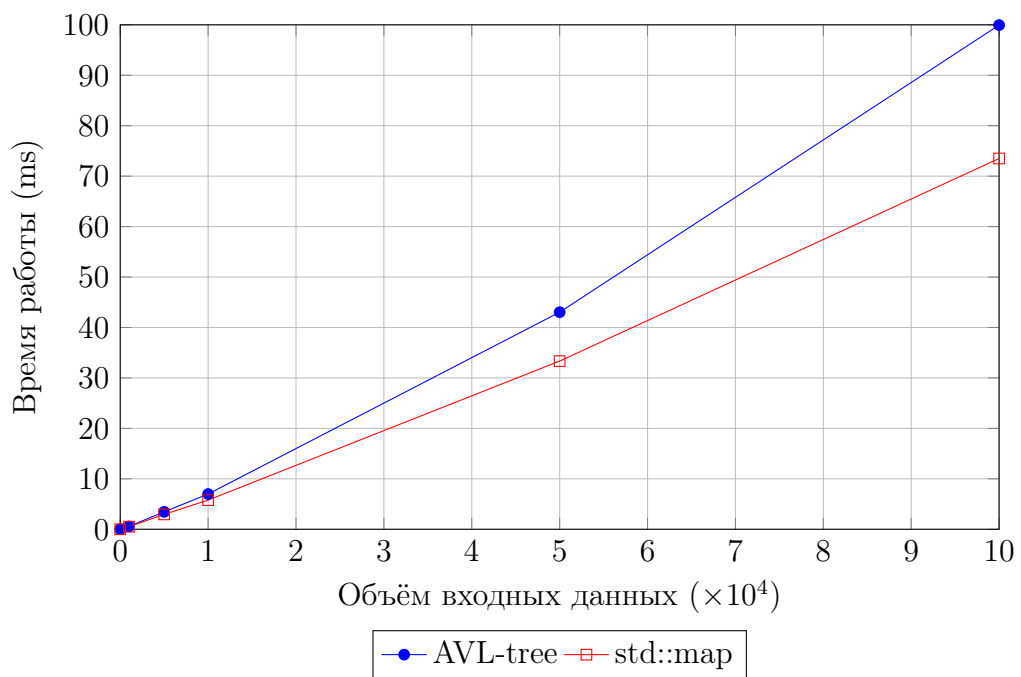


Рис. 1: Вставка

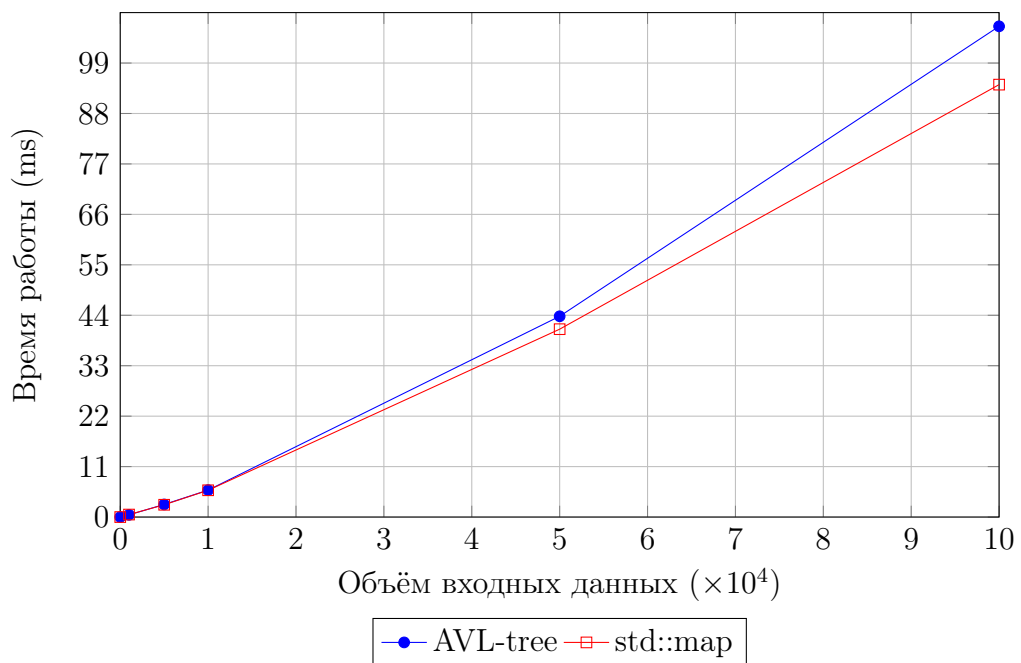


Рис. 2: Удаление

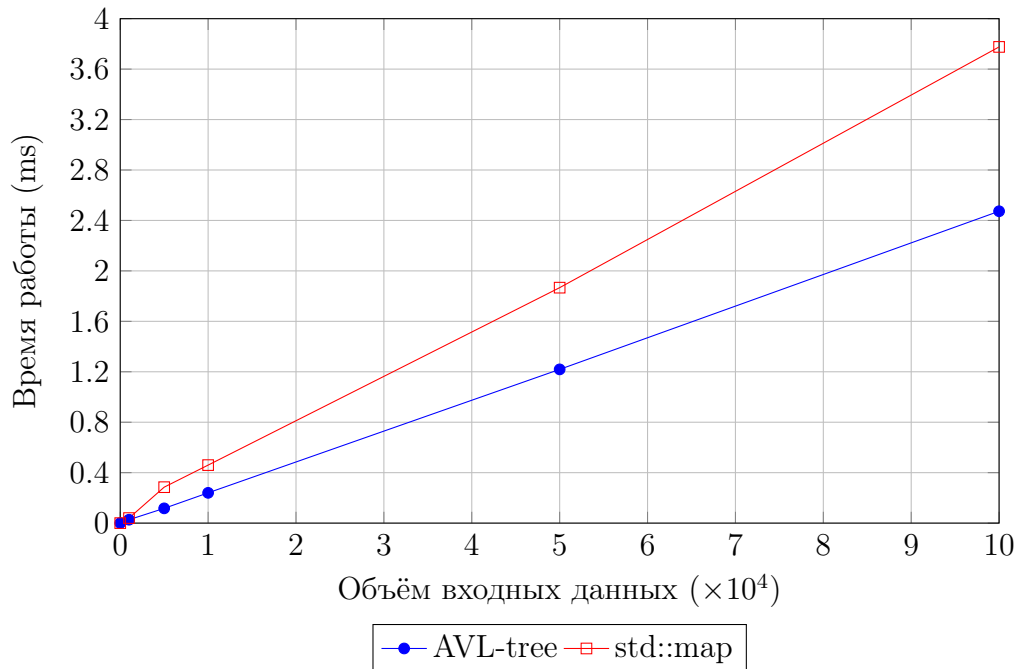


Рис. 3: Поиск

Выводы

В ходе данной лабораторной работы были изучены различные виды деревьев и реализован словарь на основе AVL-дерева. Были разобраны операции работы с данным деревом такие, как вставка, удаления и поиск узла, а также сохранение дерева файл в бинарном представлении и загрузка дерева их файла. Такая структура данных отлично подходит для хранения большого объёма данных, так как все операции выполняются до $O(\log(n))$.